

MODULO 4 CONTROL 2

Nombre: Andrés Padrón Quintana

1.- Investiga y explica que es el proceso de “poda” en arboles de decisión y como se implementa este en Python.

La poda es el proceso para reducir la complejidad de un árbol que creció demasiado, eliminando ramas poco útiles para bajar el sobreajuste y mejorar el desempeño fuera de muestra.

Se usa porque:

- Disminuye varianza y hace el modelo más estable.
- Mejora la generalización y la interpretabilidad.
- Evita que el árbol memorice ruido.

Existen dos enfoques principales: pre-poda y post-poda

Cómo se implementa en Python:

- Pre-poda: al crear el árbol, fijamos hiperparámetros como profundidad máxima y mínimo de observaciones por hoja, luego entrenamos y validamos.
- Post-poda:
 1. Entrenamos un árbol base y obtenemos la ruta de poda (una secuencia de posibles valores de `ccp_alpha`).
 2. Para cada `ccp_alpha` de esa ruta, reentrenamos el árbol y evaluamos por validación cruzada con la métrica de interés (por ejemplo, F1, ROC-AUC o RMSE si es regresión).
 3. Elegimos el `ccp_alpha` que maximiza el desempeño de validación.
 4. Entrenamos el árbol final usando ese `ccp_alpha` óptimo y reportamos su tamaño (hojas/profundidad) y su rendimiento out-of-sample.

- Al aumentar ccp_alpha , el árbol se vuelve más simple; suele mejorar hasta un punto y luego empeorar.
- El mejor ccp_alpha es el que maximiza la métrica de validación, logrando un árbol más pequeño con rendimiento igual o mejor que el no podado.

2.- Investiga y explica con tus palabras cómo funciona el algoritmo de aprendizaje supervisado “XGBoost”

XGBoost es un método de gradient boosting con árboles de decisión como modelos base. Construye un ensamble aditivo: empieza con un modelo simple y va agregando árboles pequeños que corrigen los errores del ensamble actual.

La idea central parte de:

1. Partimos de una predicción inicial (p. ej., promedio de estaturas).
2. Calculamos, para cada observación, gradientes de la función de pérdida (esto para ver qué tan mal vamos y en qué dirección corregir).
3. Ajustamos un nuevo árbol para predecir esos gradientes
4. Actualizamos la predicción sumando una fracción del nuevo árbol (learning rate o “shrinkage”).
5. Repetimos varias veces y cada árbol corrige lo que los anteriores no pudieron.

El X se debe a “eXtreme” y esto hace que:

- Regularización explícita en la función objetivo (penaliza la complejidad del árbol: número de hojas y pesos) → menos sobreajuste.
- Aproximación de segundo orden (usa gradiente y curvatura) para elegir divisiones de árbol más informadas.

3.- Investiga y explica con tus palabras como funciona el algoritmo del descenso de gradiente estocástico.

Queremos minimizar una función de pérdida (por ejemplo, error medio) ajustando los parámetros del modelo. En lugar de calcular el gradiente con

todo el conjunto de datos, se usa una observación o un minilote para aproximar el gradiente y actualizar rápido los parámetros muchas veces.

Paso a paso

- Inicializamos los parámetros del modelo
- Barajamos los datos y los recorremos por minilotes.
- Para cada minilote:
Calculamos el gradiente aproximado de la pérdida respecto a los parámetros usando solo ese minilote.
Actualizamos los parámetros moviéndonos en la dirección opuesta al gradiente: el paso = learning rate (tasa de aprendizaje).
- Cuando terminamos todas las observaciones, cerramos un epoch, volvemos a barajar y repetimos hasta que la pérdida deje de mejorar o cumplas un criterio de parada.

¿Por qué estocástico?

Porque el gradiente que se usa en cada paso está basado en una muestra parcial y contiene ruido.

Enviar solución a sergio.chavez.molotla@itam.mx

Fecha de entrega: 23 de Noviembre 2025.